GUIA TÉCNICO | 13

Fundamentos matemáticos de convolução e CNNs

Capítulo 13/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Calcular saídas de convolução, padding, stride, pooling e parâmetros.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Filtros compartilham pesos espacialmente. Em imagens, convoluções detectam padrões locais; camadas profundas combinam representações. Dimensão espacial depende de kernel, padding, dilatação e stride.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: $out = \lfloor (n + 2p - d(k-1) - 1)/s \rfloor + 1$

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Correlação cruzada, stride, padding e dilatação determinam o mapa de características; checar shapes e convenções.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
im=np.arange(25).reshape(5,5)
k=np.array([[1.,0.],[-1.,0.]])
out=np.array([[np.sum(im[i:i+2,j:j+2]*k) for j in range(4)] for i in range(4)])
print('Imagem:',im.shape,'kernel:',k.shape,'saída valid:',out.shape)
assert out.shape==(4,4)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Convolução válida, padding e saída dimensional

Correlação cruzada, stride, padding e dilatação determinam o mapa de características; checar shapes e convenções.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
image=np.arange(1,26,dtype=float).reshape(5,5)
filter=np.array([[1,0,-1],[1,0,-1],[1,0,-1]],dtype=float)
def corr_valid(X,K):
    h,w=K.shape
    return np.array([[np.sum(X[i:i+h,j:j+w]*K) for j in range(X.shape[1]-w+1)] for i in
    range(X.shape[0]-h+1)])
valid=corr_valid(image,filter)
padded=corr_valid(np.pad(image,1,mode='constant'),filter)
assert valid.shape==(3,3) and padded.shape==(5,5)
print('valid',valid.shape,'same via padding',padded.shape,'filtro
    vertical',valid[0].tolist())
```

Bibliotecas de CNN implementam correlação cruzada na prática; não confundir com convolução matemática que inverte o kernel.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Quantos parâmetros tem Conv2D com kernel 3×3, 3 canais de entrada e 16 filtros, com bias?

Resposta: 3×3×3×16 + 16 = 448 parâmetros.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências,

confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide